



**Habib University**  
shaping futures

Habib University

# OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 02 - Part 2

Arrays and C-Strings





## THIS WEEK'S AGENDA

- 01. Arrays in C++ (C-String and Array of Strings)
- 02. Revisiting functions in C++
- 03. Passing Arrays to functions
- 04. An even shorter way of writing conditions





# ARRAYS IN C++





## 1D ARRAYS IN C++

- An array is a **data type** containing elements of the **same type (homogeneous)** of data, stored in **contiguous memory locations**.
- We declare an array by specifying the data type, followed by the name of the array, followed by its size in [square brackets]. For example:
  - *int list[4]* declares an array of 4 integers called *list*.
- Each element of the array can be accessed by its subscripts.
- Subscripts start from 0 to size - 1 (in this case from 0 to 3).

```
//Some array
//initialization options

int a[4];
//declares an
//array of size 4

int a[] = {20, 40};
//declares an array
//with values
//size is implicit

int a[];
//illegal; size must
//be specified
```



## 2D ARRAYS IN C++

- Arrays in C++ can be one-dimensional or multi-dimensional.
- A 2D array can be declared in the following way:
  - `double m[3][4];` // this declares an array of size 12 i.e 3 rows and 4 columns.
- The array is indexed as follows, where m is name of the array:

|       | Column 0             | Column 1             | Column 2             | Column 3             |
|-------|----------------------|----------------------|----------------------|----------------------|
| Row 0 | <code>m[0][0]</code> | <code>m[0][1]</code> | <code>m[0][2]</code> | <code>m[0][3]</code> |
| Row 1 | <code>m[1][0]</code> | <code>m[1][1]</code> | <code>m[1][2]</code> | <code>m[1][3]</code> |
| Row 2 | <code>m[2][0]</code> | <code>m[2][1]</code> | <code>m[2][2]</code> | <code>m[2][3]</code> |

```
//LEGAL WAYS TO  
//DECLARE A 2-D ARRAY  
int m[][4] = {{1, 2, 3, 4},  
              {5, 6, 7, 8},  
              {9, 10, 11, 12}};  
  
int m[3][4];
```





## ARRAYS IN C++

**There is no limit to array dimensions.  
You can make an n-D array by adding as many  
square brackets as you'd like provided that you  
have enough memory to hold it.**





## PROCESING ARRAYS IN C++

- There are different ways to populate an array with elements:
  - For loop: Can be used to specifically set each element of the array.
  - Initializer list: Can be used to specify each element at the time of array declaration.
    - `int n[5] = {1, 2, 3, 4, 5};`
      - If not enough initializers, the rightmost elements are 0. If too many, there's a syntax error.
    - If array size is omitted, initializers determine the size:
      - `int n[5] = {1, 2, 3, 4, 5};`
        - Since there are 5 initializers, the size of the array is 5.





## PROCESING ARRAYS IN C++

- We also use loops to process arrays in C++. To print a 1D array:

```
int a[] = {1, 2, 3, 4};  
for (int i = 0; i < 5; i++) {  
    std::cout << a[i] << " ";  
}
```

- To print a 2D array we use nested for loops:

```
int a[][2] = {{1, 2}, {3, 4}};  
for (int i = 0; i < 2; i++) {  
    for (int j = 0; j < 2; j++) {  
        std::cout << a[i][j] << " ";  
    }  
}
```







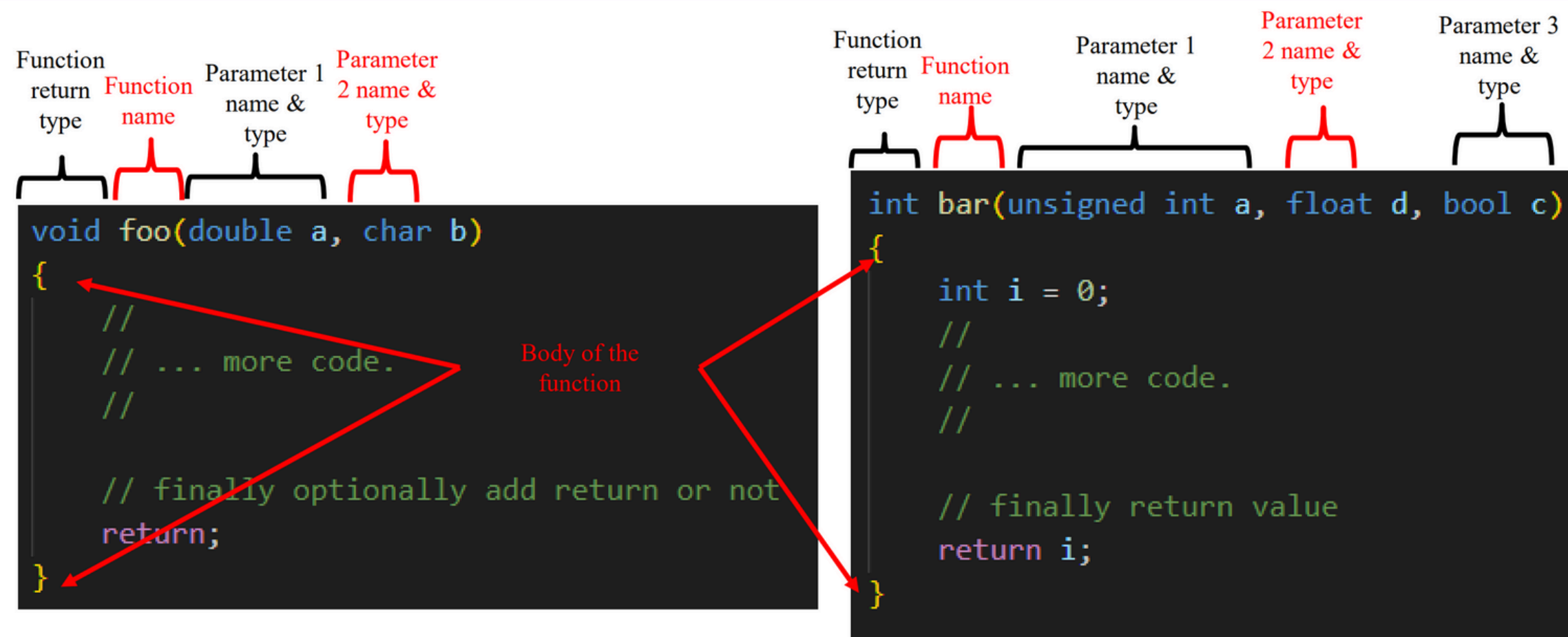
# REVISITING FUNCTIONS





## FUNCTIONS IN C++

- The syntax of a C++ function is as follows:





## DECLARATIONS VS DEFINITIONS

- A function without its body is called a function declaration.

Note semicolon



```
double computeArea(double a, double b);  
  
int main()  
{  
    double area = computeArea(2., 5.);  
    return 0;  
}
```





## DECLARATIONS VS DEFINITIONS

- We are used to seeing declarations and definitions of functions in one go.

```
// function declared and defined in one shot
double computeArea(double a, double b)
{
    //
    // ... more code.
    //

    // finally optionally add return or not
    return 0.;
}

int main()
{
    // compute area at least declared prior to the call
    double area = computeArea(2., 5.);
    return 0;
}
```





## DECLARATIONS VS DEFINITIONS

```
int main()
{
    double area = computeArea(2., 5.);
    return 0;
}

double computeArea(double a, double b)
{
    //
    // ... more code.
    //

    // finally optionally add return or not
    return 0.;
}
```

```
double computeArea(double a, double b);

int main()
{
    double area = computeArea(2., 5.);
    return 0;
}

double computeArea(double a, double b)
{
    //
    // ... more code.
    //

    // finally optionally add return or not
    return 0.;
}
```

Which of these two codes will compile?







## DECLARATIONS VS DEFINITIONS

**But why do we want to declare first and then define?**





## DECLARATIONS VS DEFINITIONS

- We usually “declare” functions in the header files. (.hpp files)
- And those functions are “defined” in the source files. (.cpp files)
- Advantages of separating declaration and definition:
  - Reduces code pollution and hence improves readability.
  - Improves compile time.
  - Enables you to hide implementation details.

***We will be encountering separating declarations and definitions in different files in the future!***





# PASSING ARRAYS TO FUNCTIONS







## PASSING ARRAYS TO FUNCTIONS

- We pass the array along with its size to a function.

```
3 void printArr(int arr[], const size_t arrSize);
4
5 int main() {
6     int myArray[] = {1, 2, 3, 4, 5};
7     printArr(myArray, sizeof(myArray) / sizeof(myArray[0]));
8     return 0;
9 }
10
11 void printArr(int arr[], size_t arrSize) {
12     for (size_t i = 0; i < arrSize; i++) {
13         std::cout << arr[i] << " ";
14     }
15     std::cout << std::endl;
16 }
```





## PASSING ARRAYS TO FUNCTIONS

- Task: 1-D Array Addition and Multiplication.
  - Write a program in C++ that reads in as inputs two arrays of one dimension using a function read. It then computes the element wise sum and product of these two arrays using the functions sum and product. Make sure to use suitable parameter lists for the functions.

***Note: In a function declaration all (but one) of the subscripts of the array size must be stated. For example:***

- ***void func(int a [][])*** is not legal, but ***void func(int a[][5])*** is valid.

***This scales to arrays of higher dimensions as well.***





# 1D ARRAYS AS STRINGS





## 1D ARRAYS AS STRINGS

- There are 2 kinds of strings used in C++
  - Objects of the string class.
  - 1D arrays of type *char*, also known as C-strings.
- A C-string can be defined as an array of characters.
- The length of the string has to be declared, and is usually the maximum possible size that the string is supposed to have.
- For example:

```
const int max = 80;  
char cString[max];
```





## C-STRINGS: EXAMPLE

```
#include <iostream>

using namespace std;

int main(void) {
    const int MAX = 80;
    char str[MAX];

    cout << "Enter the string: ";
    cin >> str;

    cout << "The string you entered: " << str << endl;
    return 0;
}
```







## C-STRINGS: EXAMPLE

- Lets say we enter the string “picture”. It will be saved in the 1D array in the following way:

|   |   |   |   |   |   |   |    |  |
|---|---|---|---|---|---|---|----|--|
| p | i | c | t | u | r | e | \0 |  |
|---|---|---|---|---|---|---|----|--|

- Each character occupies 1 byte of memory.
- C-strings must terminate with a byte containing 0, which is represented by the null character constant ‘\0’ (ASCII value of 0).
- The << insertion operator displays characters until it encounters the null character.





## C-STRINGS PRACTICES

- To avoid buffer overflow (***what is buffer?***) while reading strings, use the setw operator. (Go over Page 292, Lafore)
- Strings can be initialized by directly assigning them to the array:
  - `char str[] = "Hello World";`
- To read text via cin which contains blanks, we use `cin.get()`
- To copy a string character by character, we can use a for-loop.





## C-STRINGS: LIBRARY FUNCTIONS

- Some important C-string functions are given below and can be utilized via the `#include <cstring>` file.

**strcat()**

**Appends one C-style string to the end of another.**

**strcmp()**

Compares the ASCII values of characters in two C-style strings to determine which string has a higher value.

**strcpy()**

Copies the characters of a C-style string into the memory of another string.

**strlen()**

Returns the length of a C-style string







# THE TERNARY OPERATOR





## THE TERNARY OPERATOR

- The ternary operator is the shortest way of writing conditional statements.
- It can be used as an **inline conditional statement** in place of if-else to execute some conditional code.
- The syntax of the ternary (or conditional) operator is:
  - `expression ? if_true : if_false`
  - For example:
    - `(48 % 2 == 0) ? "48 is even" : "48 is odd";`

**Lets demo this live!**





# SUMMARY





## SUMMARY

- This week we recapped OOP vs Procedural Paradigm and Loops
- We then covered:
  - Switch statements
  - Arrays (1D, 2D and n-D)
  - Arrays as strings (C-strings)
  - Passing arrays as parameters to functions
- Note: C++ does not have methods to check for array bounds, i.e accessing an element beyond the array boundaries is not checked and hence some care should be exercised.





## READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 07 and 10







## ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

